



AUDIT REPORT

June 2026

For



Orochi

Table of Content

Executive Summary	03
Number of Security Issues per Severity	05
Summary of Issues	06
Checked Vulnerabilities	07
Techniques and Methods	09
Types of Severity	11
Types of Issues	12
Severity Matrix	13
Architecture Overview	14
 Medium Severity Issues	16
1. ccipMintedSupply Can Drift From BSC-Side Liquidity Due To Saturating Decrement	16
2. ETH-to-BSC Burns Can Proceed Without Local Proof of BSC Release Liquidity	17
3. Public Uncapped deposit() Can Expand wON Supply and Route Demand Beyond Intended Seeded Liquidity	18
 Informational Issues	19
4. MAX_CCIP_MINTED Is Not a Total wON Supply Cap	19
Functional Tests	20
Threat Model	21
Automated Tests	28
Closing Summary & Disclaimer	29



Executive Summary

Project Name	Orochi Network
Protocol Type	Chainlink CCIP Bridge
Project URL	https://orochi.network/
Overview	The reviewed design deploys WrappedON (wON) on Ethereum. BSC ON is locked/released by a BSC CCIP pool. Ethereum wON can be minted by CCIP when BSC ON is locked, or minted by depositing canonical ETH ON into the wrapper reserve. The latest client clarification states that both ETH ON and BSC ON are canonical tokens, with an intended aggregate supply model around 700M tokens
Audit Scope	The scope of this audit was to analyze the Wrapped ON Token Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/orochi-network/bridge/blob/main/src/WrappedON.sol
Branch	main
Contracts in Scope	contracts/WrappedON.sol
Commit Hash	b9de6da
Language	Solidity
Blockchain	BSC, Ethereum
Method	Manual Analysis, Threat Modelling
Review 1	21st - 26th May, 2026
Updated Code Received	3rd June, 2026
Review 2	4th June, 2026
Fixed In	681e53f

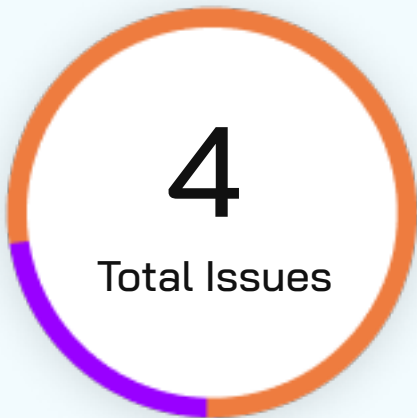


Verify the Authenticity of Report on QuillAudits Leaderboard:

<https://www.quillaudits.com/leaderboard>



Number of Issues per Severity



Critical	0 (0%)
High	0 (0%)
Medium	3 (75%)
Low	0 (0%)
Informational	1 (25%)

		Severity				
		Critical	High	Medium	Low	Informational
Issues	Open	0	0	0	0	0
	Acknowledged	0	0	0	1	3
	Partially Resolved	0	0	1	0	0
	Resolved	0	1	2	0	1



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	ccipMintedSupply Can Drift From BSC-Side Liquidity Due To Saturating Decrement	Medium	Resolved
2	ETH-to-BSC Burns Can Proceed Without Local Proof of BSC Release Liquidity	Medium	Partially Resolved
3	Public Uncapped deposit() Can Expand wON Supply and Route Demand Beyond Intended Seeded Liquidity	Medium	Resolved
4	MAX_CCIP_MINTED Is Not a Total wON Supply Cap	Informational	Acknowledged



Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations
Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls



✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Using throw

✓ Using inline assembly

✓ Style guide violation

✓ Unsafe type inference

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Implementation of ERC standards
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.



Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.



Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ **Critical: Immediate and Catastrophic Impact**

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ **High (H): Significant Risk of Major Loss or Compromise**

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ **Medium (M): Potential for Moderate Harm Under Specific Circumstances**

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ **Low (L): Minor Imperfections with Limited Repercussions**

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ **Informational (I): Opportunities for Improvement, Not Immediate Risks**

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.



Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.



Severity Matrix

		Impact		
		High	Medium	Low
Likelihood	High	Critical	High	Medium
	Medium	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.



Architecture Overview

The client originally described BSC ON as the only canonical token with a 100M supply cap. That premise has changed. The latest stated expectation is:

- ETH ON and BSC ON are both canonical tokens.
- The team does not intend to change the existing token contracts or migrate all supply from one chain to another.
- wON is an Ethereum-side bridge/liquidity receipt that can be backed by ETH ON reserve and BSC ON locked in the CCIP pool.
- The team expects to seed ETH ON liquidity into WrappedON, initially around 10M and potentially up to 100M, with CCIP rate limits to observe behavior and reduce early-stage risk.
- If BSC pool liquidity is unavailable, ETH-to-BSC redemption is expected to be unavailable until liquidity exists.

Under this updated model, the reviewed contract is not a pure BSC-backed wrapped token. It is a hybrid liquidity wrapper. That can be acceptable, but the contract and documentation need to match that model.

Route-Specific Liquidity Constraints

The bridge has two different redemption routes:

wON -> ETH ON:

requires `ETH_ON.balanceOf(WrappedON) >= amount`

wON -> BSC ON:

requires `BSC_ON.balanceOf(BSC_LockReleaseTokenPool) >= amount`

The Ethereum wrapper cannot directly read the BSC pool's balance. Therefore, ETH-to-BSC safety must be enforced by CCIP pool configuration, rate limits, off-chain monitoring, runbooks, and liquidity management.

Corrected Aggregate Supply Invariant

Let:

$E = \text{ETH_ON.totalSupply}()$

$ER = \text{ETH_ON.balanceOf(WrappedON)}$

$B = \text{BSC_ON.totalSupply}()$

$BL = \text{BSC_ON.balanceOf(BSC_LockReleaseTokenPool)}$

$W = \text{WrappedON.totalSupply}()$

If both ETH ON and BSC ON are canonical and fixed supply, the aggregate circulating-supply invariant is:

$(E - ER) + (B - BL) + W \leq 700M$



If $E + B == 700M$, this simplifies to:
 $ER + BL \geq W$

This is equivalent to the modeled backing invariant:
 $reserveON_ETH + lockedON_BSC \geq totalSupply(wON)$

The important limitation is that this only proves aggregate backing. It does not prove that a specific exit route has liquidity.



Medium Severity Issues

ccipMintedSupply Can Drift From BSC-Side Liquidity Due To Saturating Decrement

Resolved

Path

contracts/WrappedON.sol

Function Name

`_decrementCcipMinted()`

Description

All CCIP burn functions call `_decrementCcipMinted(amount)`, which saturates to zero:

```
uint256 current = ccipMintedSupply;
newSupply = amount >= current ? 0 : current - amount;
ccipMintedSupply = newSupply;
```

This allows a burn amount greater than `ccipMintedSupply` to succeed, setting the counter to zero instead of reverting.

Under the updated hybrid model, deposit-backed wON may intentionally be burned for ETH-to-BSC transfers. That means a strict `amount <= ccipMintedSupply` rule may not match the team's intended liquidity behavior. However, the current counter should not be presented as a reliable source of truth for BSC locked liquidity or live CCIP exposure once deposit-backed burns can affect it.

Impact

`ccipMintedSupply` can understate BSC-side route exposure or refill CCIP mint headroom after deposit-backed burns.

Recommendation

Pick one accounting model:

- If `ccipMintedSupply` is intended to track only BSC-backed mint exposure, do not decrement it for deposit-backed burns and revert when a BSC-backed burn exceeds the tracked amount.
- If `ccipMintedSupply` is intended only as a soft cap headroom counter, rename it and update documentation so it is not used as a BSC liquidity proxy.

In either case, BSC source-of-truth liquidity should be monitored as:
`BSC_ON.balanceOf(BSC_LockReleaseTokenPool)`



Medium Severity Issues

ETH-to-BSC Burns Can Proceed Without Local Proof of BSC Release Liquidity

Partially Resolved

Path

contracts/WrappedON.sol

Function Name

`burn(...), burnFrom()`

Description

The Ethereum-side burn functions destroy wON when called by the CCIP pool. They do not and cannot verify that the BSC LockReleaseTokenPool has enough BSC ON to release on the destination chain.

This is a known limitation of the selected design: the Ethereum contract cannot synchronously read BSC liquidity. However, this limitation must be treated as an explicit trust assumption and enforced operationally. If a source-side burn is accepted while BSC liquidity is unavailable, the user's wON can be burned while the BSC release fails or waits for manual liquidity restoration.

Impact

Users can experience stuck or delayed bridge transfers if the source burn executes but the destination pool lacks releasable BSC ON.

Likelihood:

High

Recommendation

The bridge should enforce or operationalize the following rule:
maximum ETH-to-BSC transfer over any time window $\leq$ BSC pool available liquidity - safety buffer



Medium Severity Issues

Public Uncapped deposit() Can Expand wON Supply and Route Demand Beyond Intended Seeded Liquidity

Resolved

Path

contracts/WrappedON.sol

Function Name

`deposit()`

Description

The team stated that it may seed a limited amount of canonical ETH ON into WrappedON, potentially starting around 10M and scaling up to 100M. The current deposit() function is public and uncapped. Any ETH ON holder can deposit ETH ON and mint wON 1:1.

This does not necessarily break the updated aggregate supply invariant, because deposited ETH ON is removed from ETH circulating supply and replaced by wON:

$(\text{ETH_ON.totalSupply} - \text{ETH_ON.balanceOf(WrappedON)}) + \text{totalSupply(wON)}$

However, it can expand wON supply and therefore increase demand for BSC redemption beyond the liquidity and rate limits the team intended to support.

Impact

If users can freely convert ETH ON into wON, they may attempt to bridge wON to BSC faster than BSC liquidity can support. This turns a limited-liquidity bridge launch into a public, uncapped conversion path. The resulting pressure can exhaust BSC pool liquidity and increase stuck-message risk.

Recommendation

If the team wants only protocol-managed reserve liquidity, restrict deposit():

- Make deposit() role-gated for a liquidity manager.
- Add a MAX_ETH_RESERVE cap:
 $\text{ETH_ON.balanceOf(WrappedON)} \leq \text{MAX_ETH_RESERVE}$



MAX_CCIP_MINTED Is Not a Total wON Supply Cap

Resolved

Path`contracts/WrappedON.sol`**Function Name**`MAX_CCIP_MINTED`**Description**

MAX_CCIP_MINTED = 100M caps only the ccipMintedSupply counter used by the CCIP mint path. It does not cap total wONsupply because deposit() can mint wON outside that counter.

Under the latest model, this may be intentional: BSC-side liquidity is limited around the BSC supply, while ETH-side deposits can represent canonical ETH ON held in reserve.

Recommendation

Do not present MAX_CCIP_MINTED as a wON.totalSupply() cap. Use precise language: MAX_CCIP_MINTED caps the local CCIP mint counter, not aggregate wON supply.

Functional Tests

Some of the tests performed are mentioned below:

- ✓ deposit and withdraw works in a round trip
- ✓ ccipMintHeadroomUsed correctly tracks mints and burns
- ✓ saturating burn does not allow burns above the supply
- ✓ ccipAdmin is set 2-step
- ✓ ccipMintHeadroomUsed is capped at BSC liquidity cap of MAX_CCIP_MINTED (100M tokens)



Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates everything the protocol does not fully control.

1.1 External Dependencies Table

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
ETH ON token	External canonical ERC20	Held by WrappedON as reserve for deposit()and withdraw()).	ETH ON is fixed-supply, 18-decimal, lossless, non-rebasing, and not unexpectedly paused or blacklisted.	Fee-on-transfer, pause, blacklist, or supply mutation can break reserve accounting or withdrawals.
BSC ON token	External canonical ERC20	Locked and released by the BSC CCIP LockReleaseTokenPool.	BSC ON is fixed-supply and the BSC pool holds enough liquidity for configured ETH-to-BSC transfers.	Insufficient BSC liquidity causes stuck or failed destination releases.



Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
Chainlink CCIP Router / OnRamp / OffRamp	Cross-chain messaging infrastructure	Routes token transfer messages and invokes source/destination pools.	CCIP correctly validates messages, remote pools, source pools, and rate limits.	Misconfiguration or CCIP failure can mint, burn, or release incorrectly, or halt transfers.
Ethereum CCIP BurnMintTokenPool	External pool contract	Calls mint() on BSC-to-ETH and burn functions on ETH-to-BSC.	Only the audited pool receives MINTER_ROLE and BURNER_ROLE.	Pool compromise or wrong role assignment can mint unbacked wON or burn user balances.
BSC CCIP LockReleaseTokenPool	External pool / liquidity custodian	Custodies BSC ON for release to users.	Pool balance and configured limits are sufficient for accepted ETH-to-BSC traffic.	Destination liquidity exhaustion can strand source-side burns until recovery.

2. Entry & Exit Points Analysis (Function-Level)

This is the core of the threat model. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Each function gets one table.

Contract Name	Function	Category
WrappedON	constructor(l ERC20 onToken, address admin)	What this function can do Deploy wON, bind the immutable ETH ON reserve token, grant DEFAULT_ADMIN_ROLE, and set the initial CCIP admin.



Contract Name	Function	Category
WrappedON	deposit(uint256 amount)	Main invariant(s) Reserve token and wON decimals match for 1:1 accounting. Access level Deployment only. What this function can do Pull ETH ON from caller and mint equal wON to caller. What this function cannot/should not do It does not prove BSC release liquidity and currently does not enforce an ETH reserve cap. Main invariant(s) ETH reserve increases by the wON minted through this path. Access level Any ETH ON holder with allowance.
WrappedON	withdraw(uint256 amount)	What this function can do Burn caller's wON and transfer equal ETH ON reserve to caller What this function cannot/should not do It cannot preserve provenance of deposit-backed versus CCIP-backed wON; all wON is fungible. Main invariant(s) ETH reserve must cover the withdrawal amount; wON.totalSupply() decreases. Access level Any wON holder.

Contract Name	Function	Category
WrappedON	mint(address account, uint256 amount)	What this function can do Mint wON for CCIP inbound BSC-to-ETH flow and increment ccipMintedSupply up to MAX_CCIP_MINTED. What this function cannot/should not do It must not be callable by anyone except the audited CCIP pool and should not mint without a corresponding BSC-side lock. Main invariant(s) ccipMintedSupply <= MAX_CCIP_MINTED; CCIP mint should correspond to BSC locked ON. Access level MINTER_ROLE
WrappedON	burn(uint256 amount)	What this function can do Burn wON from msg.sender for CCIP outbound transfer. What this function cannot/should not do It cannot locally verify BSC release liquidity. Main invariant(s) Accepted burns should be bounded by destination route liquidity and rate limits. Access level BURNER_ROLE
WrappedON	burnFrom(address account, uint256 amount)	What this function can do Spend allowance and burn wON from account. What this function cannot/should not do It should not bypass allowance and cannot verify BSC liquidity.

Contract Name	Function	Category
WrappedON	setCCIPAdmin (address newAdmin)	Main invariant(s) Allowance is consumed before burn; route liquidity must exist operationally. Access level BURNER_ROLE. What this function can do Propose a new CCIP admin using a two-step handoff. What this function cannot/should not do It must not set zero, the current admin, or the token contract itself as pending admin. Main invariant(s) Only current CCIP admin can initiate rotation. Access level Current s_ccipAdmin.
WrappedON	acceptCCIPAdmin()	What this function can do Complete CCIP admin transfer and clear pending admin. What this function cannot/should not do It must not be callable by anyone except the pending admin. Main invariant(s) CCIP admin handoff is two-step. Access level Pending CCIP admin only.

3. Asset Flow Mapping (Critical Contracts)

This section identifies where money actually lives and how it moves.

3.1 Asset-Holding Contracts

List only contracts that custody value.

Contract	Asset Type	Custodied Assets
WrappedON	ETH ON reserve	Holds canonical ETH ON deposited through deposit() or direct transfer. This reserve funds withdraw().
WrappedON	wON ERC20 accounting	Tracks fungible wON balances minted by deposit() and CCIP mint(), and burned by withdraw() or CCIP burn paths.
BSC LockReleaseTokenPool	BSC ON liquidity	Holds canonical BSC ON used to release tokens for ETH-to-BSC transfers. External to WrappedON but critical to bridge liveness.

3.2 Asset Entry & Exit Functions

This table maps money movement, which is where most exploits happen.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
WrappedON	deposit(uint256)	ETH ON from caller	wON minted to caller	Any ETH ON holder with allowance	Public and uncapped; can expand wON supply and BSC redemption demand.



Contract	Function	Asset In	Asset Out	Caller	Risk Notes
WrappedON	withdraw(uint256)	wON burned from caller	ETH ON reserve to caller	Any wON holder	Any wON holder can consume ETH reserve, regardless of mint origin.
WrappedON	mint(address,uint256)	CCIP assumption that BSC ON was locked	wON minted to receiver	MINTER_ROLEpool	Pool/role compromise can mint unbacked wON up to cap headroom.
WrappedON	burn(uint256)	wON burned from pool/caller	CCIP message expected to release BSC ON	BURNER_ROLEpool	Cannot locally verify BSC liquidity; relies on rate limits and monitoring.
WrappedON	burnFrom(address,uint256)	Allowance and wON burned from account	CCIP message expected to release BSC ON	BURNER_ROLEpool	Safer than arbitrary burn but still depends on destination liquidity.
WrappedON	Direct ETH ON transfer	ETH ON	None immediately	Any ETH ON holder	Increases reserve without Wrapped event; monitors must read token balance and ERC20 transfers.
BSC LockRelease TokenPool	BSC-to-ETH lock	BSC ON	CCIP message to mint wON on Ethereum	CCIP pool/router	Lock must happen before Ethereum mint() for aggregate backing.

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
BSC LockRelease TokenPool	ETH-to-BSC release	CCIP burn proof from Ethereum	BSC ON to recipient	CCIP off- ramp/ pool	Release can fail or queue if pool liquidity is insufficient.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.



Closing Summary

In this report, we have considered the security of Orochi Network. We performed our audit according to the procedure described above.

3 Medium and 1 Informational severity issue were found.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.



About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers.

With seven years of expertise, we've secured over 1400 projects globally, averting over \$3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

**8+**

Years of Expertise

1M+

Lines of Code Audited

50+

Chains Supported

1500+

Projects Secured

Follow Our Journey



AUDIT REPORT

June 2026

For



Orochi



Canada, India, Singapore, UAE, UK

www.quillaudits.com

audits@quillaudits.com